ExamCell: An Automated Exam Timetable Scheduling System Using Backtracking Constraint Satisfaction

*A Java Mini-Project Report

Team Member 1
Department of Computer Engineering
MIT World Peace University
Pune, India
1032230597

Team Member 2
Department of Computer Engineering
MIT World Peace University
Pune, India
1032230316

Team Member 3
Department of Computer Engineering
MIT World Peace University
Pune, India
1032230642

Team Member 4
Department of Computer Engineering
MIT World Peace University
Pune, India
1032230784

***Abstract*—Preparing examination timetables manually is often a difficult and time-consuming task for educational institutions. It becomes even more challenging when factors like room availability, faculty assignment, student batch conflicts, and classroom capacity all need to be considered at the same time. To address this, we developed ExamCell, a Java-based exam scheduling system that automatically generates timetables using a backtracking approach based on the Constraint Satisfaction Problem model. The project was built using core Java along with Swing for the desktop interface, JDBC with MySQL for database connectivity, and Servlets/JSP for web access. The system helps administrators create conflict-free schedules and lets students view the final timetable through a web interface. Through this project, we were able to apply concepts such as object-oriented programming, multithreading, database management, and client-server application design in a practical way.**

***Index Terms*—Exam Timetable, Backtracking, Constraint Satisfaction Problem, Java, JDBC, Servlet, JSP**

I. Introduction

Every academic semester, institutions have to prepare exam timetables that fit a large number of subjects into a limited set of rooms and time slots. Doing this manually often leads to clashes, such as two exams being scheduled at the same time for the same student batch, or a classroom being assigned to more than one exam. In some cases, faculty members may also be overbooked, which makes the process even harder to manage.

To solve this problem, we developed ExamCell, a simple but practical exam scheduling system. The main goal of the project was to automate timetable generation and reduce the manual effort required from administrators. At the same time, the project also gave us a chance to work with important Java concepts such as OOP, multithreading, JDBC, and web development using Servlets and JSP.

II. System Architecture

ExamCell follows a multi-tier structure so that the different parts of the application remain separated and easier to manage. The system is divided into three main layers:

- **Data Layer**: This stores details about subjects, classrooms, faculty members, time slots, and final allocations in a MySQL database.
- **Application Logic Layer**: This part contains the main scheduling engine written in Java. It reads the stored data, runs the scheduling algorithm, and prepares the timetable.
- **Presentation Layer**: This includes a Swing-based desktop interface for administrators and a JSP-based web interface for students and faculty members.

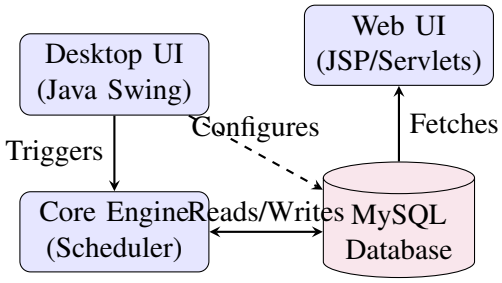


Fig. 1. ExamCell Multi-Tier System Architecture

The main entities in the system are represented using simple object-oriented classes such as `Subject`, `Classroom`, `Faculty`, `TimeSlot`, and `Allocation`. A common `DatabaseHelper` class is used to manage JDBC connections and database operations.

III. Methodology and Implementation

A. Problem Formulation as a CSP

We treated the timetable generation problem as a Constraint Satisfaction Problem. In this setup, each exam acts as a variable, while the possible combinations of time slots, classrooms, and faculty members form the domain. The final timetable must satisfy all the required constraints.

The main constraints used in the system are:

- **Room Availability**: A room can be used for only one exam in a given time slot.
- **Faculty Availability**: A faculty member can supervise only one exam at a time.
- **Batch Overlap**: Subjects from the same department and semester should not be scheduled in the same slot.
- **Capacity Constraint**: The selected classroom must have enough seating capacity for the number of students registered for that subject.

B. Backtracking Algorithm

To generate the timetable, the system uses a recursive backtracking algorithm. It starts by selecting a subject and then tries different combinations of time slots, rooms, and faculty members. If a combination does not satisfy the constraints, the system rejects it and tries another one.

The process works in the following way:

1) Select the next subject that has not been scheduled.
2) Try all possible combinations of time slot, classroom, and faculty.
3) Check whether the combination is valid using the `isValid()` method.
4) If it is valid, assign it temporarily and move to the next subject.
5) If the later step fails, remove the assignment and try another option.
6) If no valid combination is found, the algorithm backtracks and continues from the previous step.

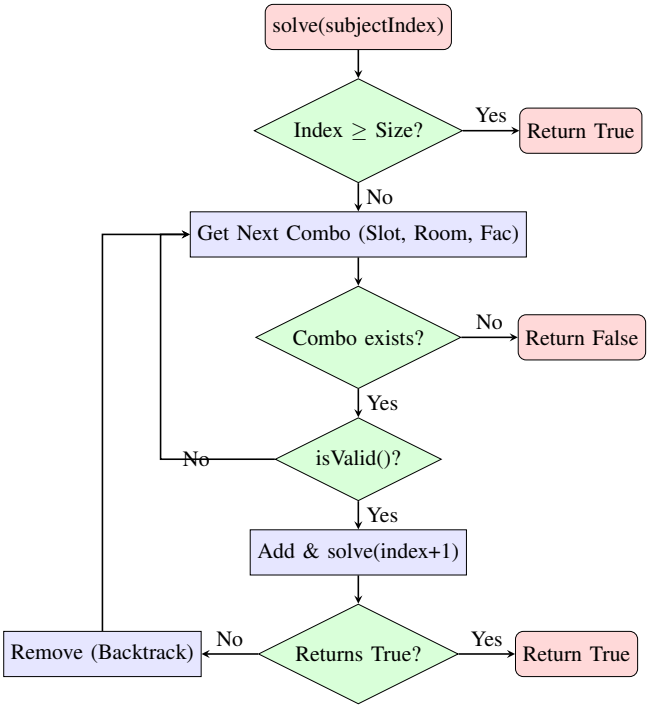


Fig. 2. Backtracking Algorithm Flowchart

C. Multithreading and UI Responsiveness

Since backtracking can take time when the number of subjects grows, we used a background thread so that the Swing interface does not freeze while the timetable is being generated. Once the scheduling is complete, the results are updated on the main UI thread using `SwingUtilities.invokeLater()`. The generated schedule is also saved to the database after completion.

D. Web Deployment

A servlet named `ViewTimetableServlet` handles the request for viewing the timetable. It connects to the database through `DatabaseHelper`, runs a SQL join query to collect the required records, and sends the data to a JSP page for display. This makes the timetable easy to access through a browser.

IV. Results and Discussion

The system was tested using a sample dataset that included 6 subjects from two departments, 3 classrooms, 4 faculty members, and 4 time slots. When the algorithm was run, it successfully produced a timetable without any clashes.

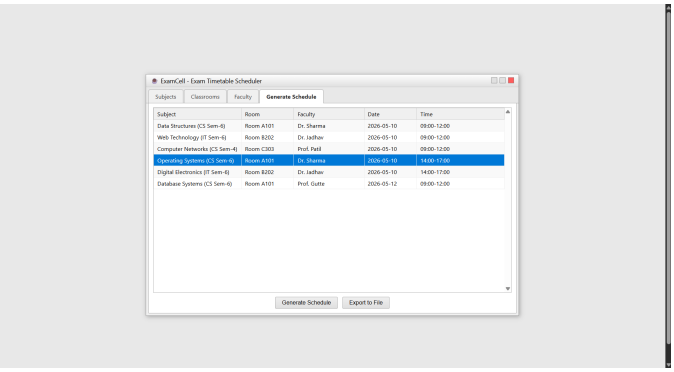


Fig. 3. The ExamCell Java Swing desktop application displaying the successfully generated schedule for administrators.

Some of the observations were:

- No two subjects from the same CS Semester 6 batch were placed in the same time slot.
- CS and IT exams could run at the same time as long as different rooms and faculty members were available.
- The final timetable was exported into a text file using `BufferedWriter`.
- JDBC worked reliably for storing the timetable and making it available through the web interface immediately after generation.

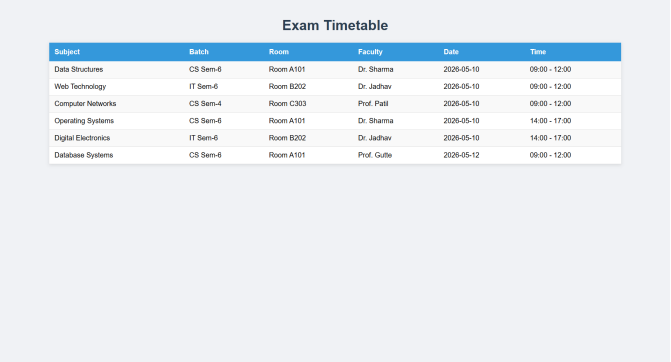


Fig. 4. The web-based interface allowing students to view the finalized timetable.

Overall, the output showed that the system was able to handle the basic scheduling rules correctly and produce a usable timetable for the given dataset.

V. Conclusion

The ExamCell project successfully demonstrates an automated approach to exam timetable generation. By using backtracking and constraint satisfaction, the system is able to reduce conflicts and create a valid schedule within the given limits. The project also helped us apply several core Java concepts in a single working application, including object-oriented programming, multithreading, JDBC, and servlet-based web development.

One possible improvement for the future would be to include smarter search techniques such as the Minimum Remaining Values heuristic or to explore genetic algorithms for larger and more complex datasets.

References

[1] R. Dechter, *Constraint Processing*, Morgan Kaufmann, 2003.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Prentice Hall, 2009.

[3] Oracle, "Java SE Documentation."

[4] MySQL, "MySQL Connector/J Developer Guide."